Task 1: Install Terraform on Windows PC

1. Objective
To install Terraform on a Windows PC, configure the system PATH, and verify the installation using Git Bash.

2. Purpose
Terraform is an Infrastructure as Code (IaC) tool developed by HashiCorp. It allows DevOps engineers to define, provision, and manage infrastructure using configuration files.

Installing Terraform is the first step toward automating infrastructure deployment and integrating Terraform with Jenkins CI/CD pipelines.

3. Clear Steps

Step 1: Download Terraform

1. Open the official HashiCorp installation page: https://developer.hashicorp.com/terraform/install
2. Select the Windows AMD64 package if using a 64-bit Windows PC.
3. Download the ZIP archive.

Step 2: Extract Terraform

1. Create the folder C:\Terraform.
2. Extract the downloaded ZIP archive.
3. Confirm that terraform.exe exists at:

C:\Terraform\terraform.exe

Step 3: Configure the PATH

1. Open Windows Search.
2. Search for **Edit the system environment variables**.
3. Open **Environment Variables**.
4. Under User variables, select Path and click Edit.
5. Add the following entry:

C:\Terraform

6. Save the changes and reopen Git Bash.

Step 4: Verify the Terraform installation

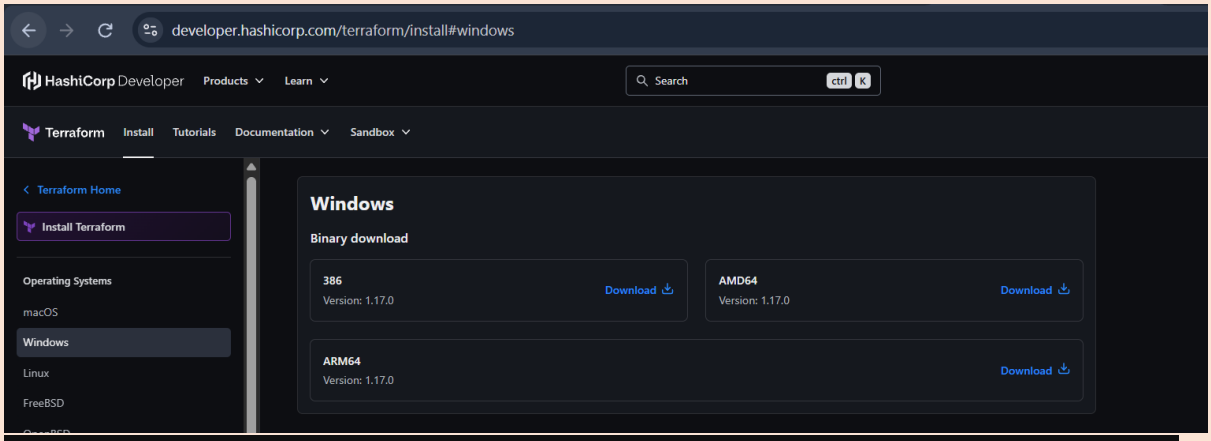
Execute the following command in Git Bash: terraform -version

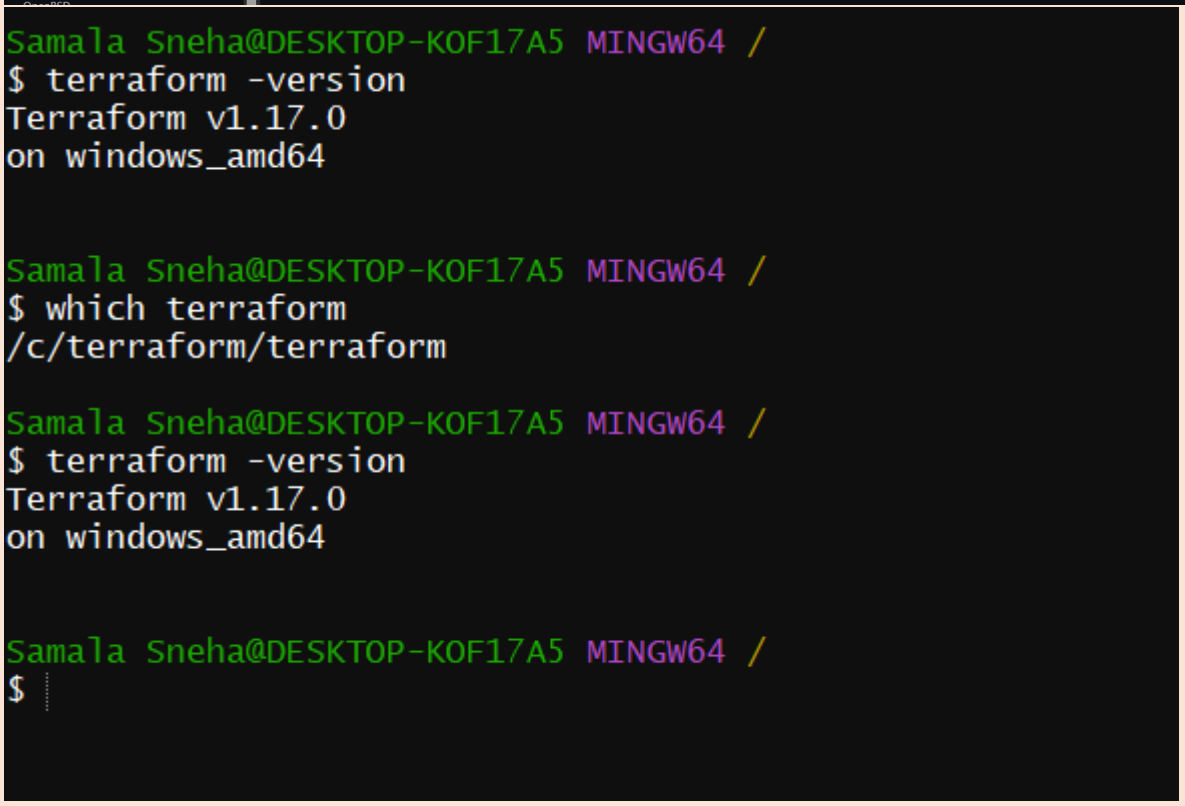
Observed output:
Terraform v1.14.3
on windows_amd64

Step 5: Verify the executable location

Execute: which terraform
Observed output: /c/terraform/terraform
This confirms that Git Bash can locate the Terraform executable.

Implementation:





4. Conclusion

Terraform was successfully installed on the Windows PC and verified using Git Bash. The installed version is Terraform 1.14.3 for Windows AMD64. The executable is accessible through the PATH, and the environment is ready for the next Terraform lab activities.

Task 2: Create and Execute Terraform Templates Shown in video

1. Objective
To create and execute Terraform configuration templates in Visual Studio Code using the Local and Random providers.

2. Purpose

Terraform is an Infrastructure as Code (IaC) tool used to define and manage resources through configuration files.

Visual Studio Code is used to write and edit Terraform files, while the Git Bash terminal is used to execute Terraform commands.

This practical demonstrates how Terraform creates a local file and generates a random pet name using Terraform providers.

3. Clear Steps
Step 1: Open VS Code

1. Open Visual Studio Code.
2. Create or open the Terraform project folder.
3. Create a new file named main.tf

Step 2: Write the Terraform configuration

Enter the following code in main.tf:

```
resource "local_file" "my_pet" {
  filename = "pets.txt"
  content  = "i love my snupi"
}

resource "random_pet" "sunith" {
  prefix    = "chintu"
  separator = "."
  length    = 3
}
```

Save the file using Ctrl + S.

Step 3: Understand the configuration

Resource 1: Local file

```
resource "local_file" "my_pet" {
  filename = "pets.txt"
  content  = "i love my snupi"
}
```

- resource: Declares a resource managed by Terraform.
- local_file: Identifies the resource type provided by the Local provider.
- my_pet: The Terraform resource's local name.
- filename: Specifies the file to create.
- content: Specifies the text written into the file.

Expected result: A file named pets.txt containing i love my snupi.

Resource 2: Random pet

```
resource "random_pet" "sunith" {
  prefix    = "chintu"
  separator = "."
  length    = 3
}
```

- random_pet: Generates a random pet-style name.
- sunith: The Terraform resource's local name.
- prefix: Adds chintu at the beginning of the generated name.
- separator: Uses a dot between name components.
- length: Requests three name components.

The generated name is stored in Terraform state. The exact name depends on the generated result and the resource's lifecycle.

Step 4: Initialize Terraform

Open the VS Code terminal and navigate to the directory containing main.tf.

Execute: terraform init

Purpose:

- Initializes the Terraform working directory.
- Downloads the required Local and Random provider plugins.
- Creates the .terraform directory.
- Creates or updates .terraform.lock.hcl.

Expected result: Terraform initializes successfully.

Step 5: Format and validate the configuration

Format the configuration: terraform fmt

Validate it: terraform validate

Expected result: Success! The configuration is valid.

Step 6: Review the Terraform plan

Execute: terraform plan

Terraform previews the resources it intends to create or modify.

For a fresh working directory with no existing managed resources, the plan should show the local file and random pet resources being created.

Step 7: Apply the configuration

Execute: terraform apply
Review the proposed changes and type yes when prompted.
Terraform creates the resources specified in the configuration.

Step 8: Verify the local file

Execute: cat pets.txt
Expected output: i love my snupi

You can also open pets.txt in VS Code to verify the contents.

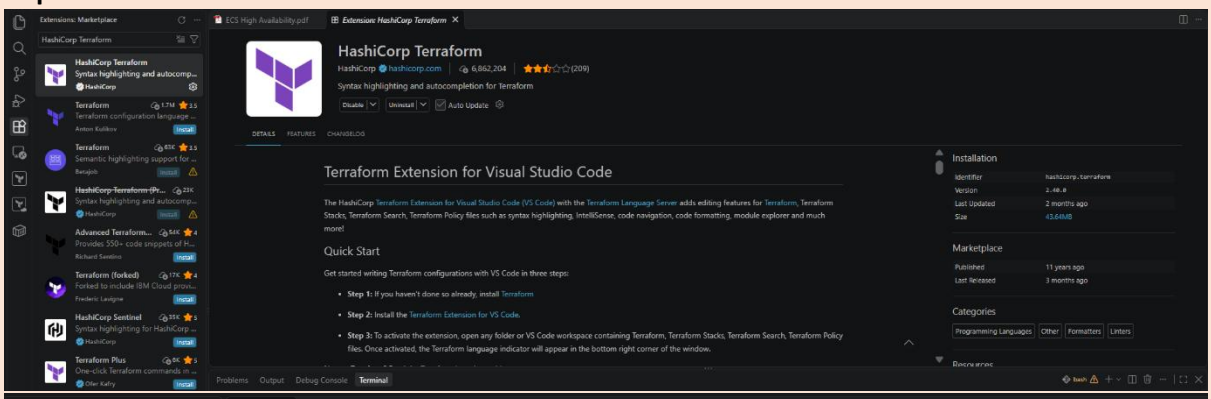
Step 9: Verify the random pet resource

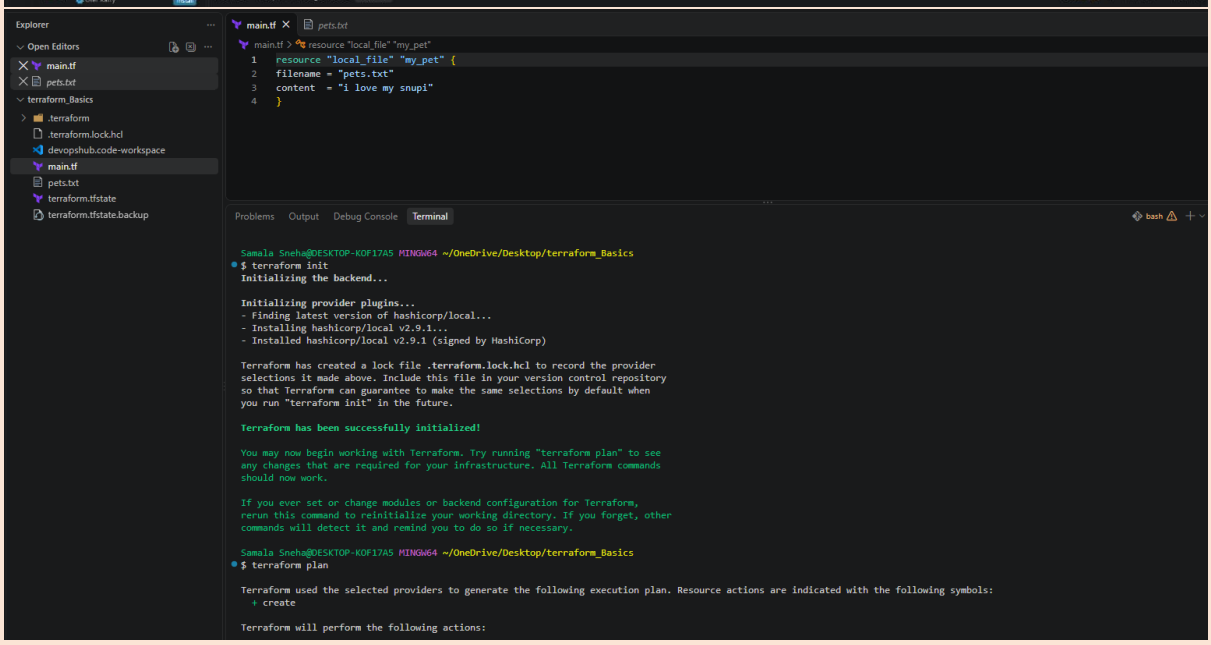
Execute: terraform state list

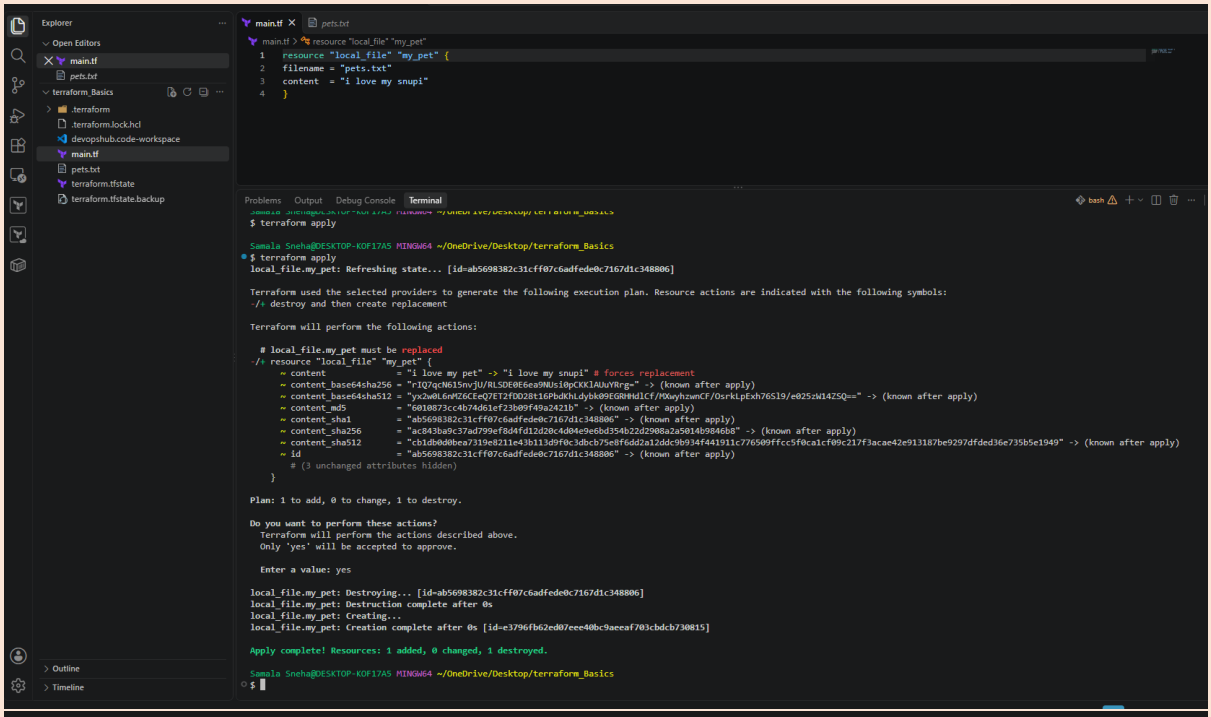
The output should include:
local_file.my_pet
random_pet.sunith

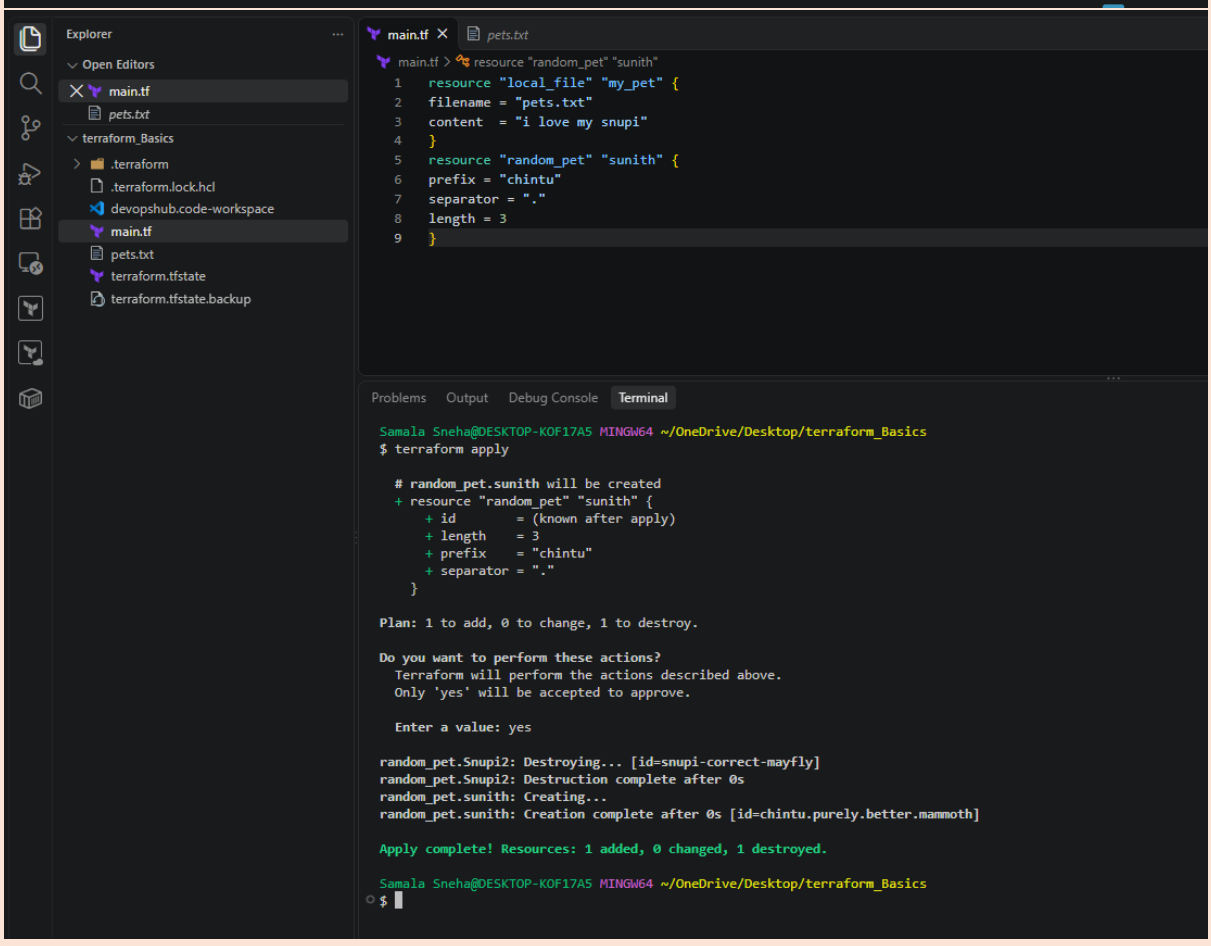
To inspect the generated name, execute:
terraform state show random_pet.sunith
Look for the id attribute, which contains the generated pet name.

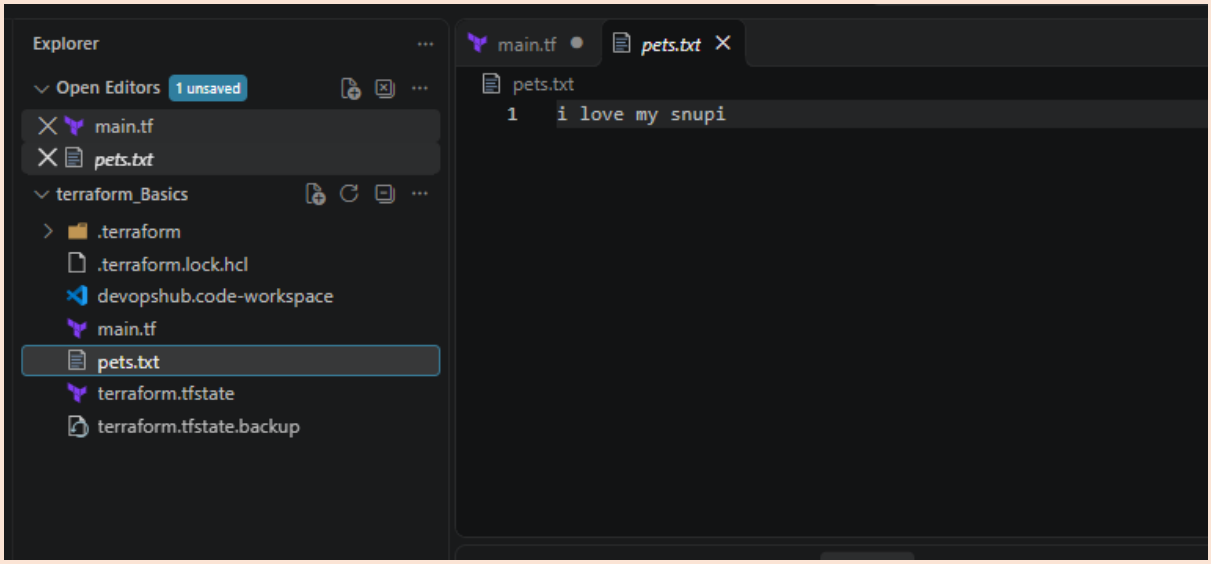
Implementation:






Explorer
Open Editors
main.tf
pets.txt
terraform_Basics
.terraform
.terraform.lock.hcl
devopshub.code-workspace
main.tf
pets.txt
terraform.tfstate
terraform.tfstate.backup
Outline
Timeline
main.tf
pets.txt
main.tf > resource "local_file" "my_pet"
1 resource "local_file" "my_pet" {
2 filename = "pets.txt"
3 content = "i love my snupi"
4 }
Problems
Output
Debug Console
Terminal
bash
$ terraform apply
Samala Sneha@DESKTOP-KOF17A5 MINGW64 ~/OneDrive/Desktop/terraform_Basics
$ terraform apply
local_file.my_pet: Refreshing state... [id=ab5698382c31cff07c6adfede0c7167d1c348806]
Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:
-/+ destroy and then create replacement
Terraform will perform the following actions:
local_file.my_pet must be replaced
-/+ resource "local_file" "my_pet" {
~ content = "i love my pet" -> "i love my snupi" # forces replacement
~ content_base64sha256 = "+1Q7qcN615nvjU/NLSDE0E6ea9NUsi0pCXKlAUuYRrg=" -> (known after apply)
~ content_base64sha512 = "yx2w0L6nMZ6CEeQ7ET2fDD28t16PbdKhLdybk09EGRHHd1CF/MXwyhzwnCF/OsrkLpExh76S19/e025zW14ZSQ==" -> (known after apply)
~ content_md5 = "6010873cc4b74d61ef23b09f49a2421b" -> (known after apply)
~ content_sha1 = "ab5698382c31cff07c6adfede0c7167d1c348806" -> (known after apply)
~ content_sha256 = "ac843ba9c37ad799ef8d4fd12d20c4d04e9e6bd354b22d2908a2a5014b9846b8" -> (known after apply)
~ content_sha512 = "cb1db00d0bea7319e8211e43b113d9f0c3dbcb75e8f6dd2a12ddc9b934f441911c776509ffcc5f0ca1cf09c217f3acae42e913187be9297dfded36e735b5e1949" -> (known after apply)
~ id = "ab5698382c31cff07c6adfede0c7167d1c348806" -> (known after apply)
(3 unchanged attributes hidden)
}
Plan: 1 to add, 0 to change, 1 to destroy.
Do you want to perform these actions?
Terraform will perform the actions described above.
Only 'yes' will be accepted to approve.
Enter a value: yes
local_file.my_pet: Destroying... [id=ab5698382c31cff07c6adfede0c7167d1c348806]
local_file.my_pet: Destruction complete after 0s
local_file.my_pet: Creating...
local_file.my_pet: Creation complete after 0s [id=e3796fb62ed07eee40bc9aeeaf703cbdcb730815]
Apply complete! Resources: 1 added, 0 changed, 1 destroyed.
Samala Sneha@DESKTOP-KOF17A5 MINGW64 ~/OneDrive/Desktop/terraform_Basics
$


Explorer
Open Editors
main.tf
pets.txt
terraform_Basics
.terraform
.terraform.lock.hcl
devopshub.code-workspace
main.tf
pets.txt
terraform.tfstate
terraform.tfstate.backup
main.tf
pets.txt
main.tf > resource "random_pet" "sunith"
1 resource "local_file" "my_pet" {
2 filename = "pets.txt"
3 content = "i love my snupi"
4 }
5 resource "random_pet" "sunith" {
6 prefix = "chintu"
7 separator = "."
8 length = 3
9 }
Problems
Output
Debug Console
Terminal
Samala Sneha@DESKTOP-KOF17A5 MINGW64 ~/OneDrive/Desktop/terraform_Basics
$ terraform apply
random_pet.sunith will be created
+ resource "random_pet" "sunith" {
+ id = (known after apply)
+ length = 3
+ prefix = "chintu"
+ separator = "."
}
Plan: 1 to add, 0 to change, 1 to destroy.
Do you want to perform these actions?
Terraform will perform the actions described above.
Only 'yes' will be accepted to approve.
Enter a value: yes
random_pet.Snupi2: Destroying... [id=snupi-correct-mayfly]
random_pet.Snupi2: Destruction complete after 0s
random_pet.sunith: Creating...
random_pet.sunith: Creation complete after 0s [id=chintu.purely.better.mammoth]
Apply complete! Resources: 1 added, 0 changed, 1 destroyed.
Samala Sneha@DESKTOP-KOF17A5 MINGW64 ~/OneDrive/Desktop/terraform_Basics
$



4. Conclusion

The Terraform template was successfully created and executed in Visual Studio Code. The Local provider was used to create a text file, and the Random provider was used to generate a pet-style name.

This exercise demonstrated how to define resources in Terraform configuration files, initialize providers, validate code, review a plan, apply changes, and verify managed resources.

The practical provides a foundation for understanding Terraform providers and integrating Terraform into automated DevOps workflows.

Task 3: Note down Below points (Terraform Init, Plan, Apply, and Provider)

1. Objective

To understand and execute the Terraform commands terraform init, terraform plan, and terraform apply, and to understand the role of Terraform providers.

2. Purpose

Terraform uses configuration files to define resources and manage their lifecycle. The init, plan, and apply commands are essential parts of the Terraform workflow. Providers allow Terraform to interact with external platforms and services.

These concepts are important for infrastructure automation, AWS provisioning, and Jenkins CI/CD integration.

3. Clear Steps

Step 1: Open the Terraform project

1. Open VS Code.
2. Open the folder containing the existing main.tf.
3. Open Terminal → New Terminal.
4. Confirm that the terminal is in the project directory.

Step 2: Understand Terraform Init

Execute: terraform init

Purpose:

- Initializes the working directory.
- Downloads required provider plugins.
- Prepares Terraform for subsequent commands.
- Creates or updates .terraform.lock.hcl.

Expected result:

Terraform has been successfully initialized!

Step 3: Understand Terraform Plan

Execute: terraform plan
Also Called as: DRY RUN

Purpose:

- Reads the Terraform configuration.
- Compares the desired configuration with the current state.
- Displays the proposed changes.

For a fresh configuration containing the two resources, the expected summary is:

Plan: 2 to add, 0 to change, 0 to destroy.

If the resources already exist and match the configuration, Terraform may report no changes.

Step 4: Understand Terraform Apply

Execute: terraform apply

Review the plan and type yes when prompted.
Purpose:

- Applies the planned configuration changes.
- Creates, updates, or removes resources as necessary.
- Updates Terraform state to record managed resources.

Verify the local file: cat pets.txt

Expected output: i love my snupi

Verify Terraform state: terraform state list

Expected resource addresses:
local_file.my_pet
random_pet.sunith

Step 5: Understand Terraform Providers

A provider is a plugin that allows Terraform to manage resources for a particular platform or service.

The current configuration uses:

1. **Local provider (hashicorp/local)** — creates and manages local files.

2. **Random provider (hashicorp/random)** — generates random values such as pet-style names.

Provider requirements can be declared in the Terraform configuration:

```
terraform {
  required_providers {
    local = {
      source = "hashicorp/local"
    }
    random = {
      source = "hashicorp/random"
    }
  }
}
```

After saving the configuration, execute: terraform init

Then validate it: terraform validate

Expected result: Success! The configuration is valid.

4. Conclusion

The Terraform init, plan, apply, and provider concepts were studied using the existing local-file and random-pet configuration.

The practical demonstrates how Terraform initializes providers, previews configuration changes, applies changes, and tracks managed resources through state.

These concepts provide the foundation for integrating Terraform into Jenkins pipelines and automating infrastructure deployment.

Task 4: Integrate a sample Terraform template with Jenkins Pipeline

1. Objective

To integrate Terraform with Jenkins and execute a sample Terraform configuration through a Jenkins Pipeline, including initialization, validation, planning, applying, and verification.

2. Purpose

Jenkins is an automation server used to build and execute CI/CD pipelines. Terraform is an Infrastructure as Code tool used to manage infrastructure through configuration files.

Integrating Terraform with Jenkins allows DevOps engineers to automate infrastructure workflows, validate configuration changes, review execution plans, and provision resources through pipeline stages.

In this practical, Jenkins and Terraform were installed on an AWS EC2 Linux instance. A Jenkins Pipeline was configured to execute a sample Terraform project using the Local and Random providers.

3. Clear Steps

Step 1: Launch an EC2 instance

1. Sign in to the AWS Management Console.
2. Open the EC2 service in the us-east-1 region.
3. Launch an Amazon Linux 2023 instance.
4. Configure a suitable instance type and storage.
5. Configure a security group allowing SSH access on port 22 from the administrator's IP and Jenkins web access on port 8080 from the administrator's IP.
6. Launch the instance and wait until it is running and its status checks pass.

Step 2: Install Java and Git

Connect to the EC2 instance and execute: sudo dnf update -y
sudo dnf install -y wget git java-21-amazon-corretto

Verify Java: java -version

Verify Git: git --version

Step 3: Install Jenkins

Configure the official Jenkins stable repository:
sudo wget -O /etc/yum.repos.d/jenkins.repo \
https://pkg.jenkins.io/rpm-stable/jenkins.repo

Import the repository signing key:
sudo rpm --import \
https://pkg.jenkins.io/rpm-stable/jenkins.io-2026.key

Install Jenkins: sudo dnf install -y Jenkins
Enable and start Jenkins: sudo systemctl enable --now jenkins

Verify the service: sudo systemctl status jenkins --no-pager

Step 4: Install Terraform

Configure the HashiCorp repository: sudo dnf install -y yum-utils shadow-utils

sudo yum-config-manager --add-repo \

https://rpm.releases.hashicorp.com/AmazonLinux/hashicorp.repo

Install Terraform: sudo dnf install -y terraform

Verify: terraform -version

The Jenkins console confirmed Terraform v1.17.0 on Linux AMD64.

Step 5: Configure Jenkins

1. Retrieve the initial administrator password:
   sudo cat /var/lib/jenkins/secrets/initialAdminPassword
2. Open Jenkins in a browser: http://YOUR_EC2_PUBLIC_IP:8080
3. Enter the initial administrator password.
4. Install the suggested plugins.
5. Create the administrator account.
6. Complete the setup wizard and open the Jenkins dashboard.

Step 6: Create a Jenkins Pipeline job

1. Open the Jenkins dashboard.
2. Click **New Item**.
3. Enter the name terraform-sample-pipeline.
4. Select **Pipeline**.
5. Click **OK**.
6. Scroll to the Pipeline section.
7. Set Definition to **Pipeline script**.

Step 7: Configure the Pipeline script

Enter the following code in the Pipeline script editor:

```
pipeline {
    agent any

    stages {
        stage('Check Tools') {
            steps {
                sh 'terraform -version'
                sh 'git --version'
            }
        }

        stage('Create Terraform File') {
            steps {
                writeFile file: 'main.tf', text: '''
terraform {
  required_providers {
    local = {
      source = "hashicorp/local"
    }
    random = {
      source = "hashicorp/random"
    }
  }
}

resource "local_file" "my_pet" {
  filename = "pets.txt"
  content  = "i love my snupi"
```

```
}

resource "random_pet" "sunith" {
  prefix    = "chintu"
  separator = "."
  length    = 3
}
'''
          }
        }

        stage('Terraform Init') {
          steps {
            sh 'terraform init -input=false'
          }
        }

        stage('Terraform Validate') {
          steps {
            sh 'terraform fmt -check'
            sh 'terraform validate'
          }
        }

        stage('Terraform Plan') {
          steps {
            sh 'terraform plan -input=false -out=tfplan'
          }
        }

        stage('Terraform Apply') {
          steps {
            sh 'terraform apply -input=false -auto-approve tfplan'
          }
        }

        stage('Verify Resources') {
          steps {
            sh 'cat pets.txt'
            sh 'terraform state list'
          }
        }
    }
}
```

Click **Save**.

Step 8: Execute the pipeline

1. Open the terraform-sample-pipeline job.
2. Click **Build Now**.
3. Open build #1.
4. Click **Console Output**.
5. Review the output of each stage.

Step 9: Verify the execution results

The console output confirmed: **Terraform initialization**
Terraform has been successfully initialized!
Terraform validation
Success! The configuration is valid.

Terraform plan Plan: 2 to add, 0 to change, 0 to destroy.

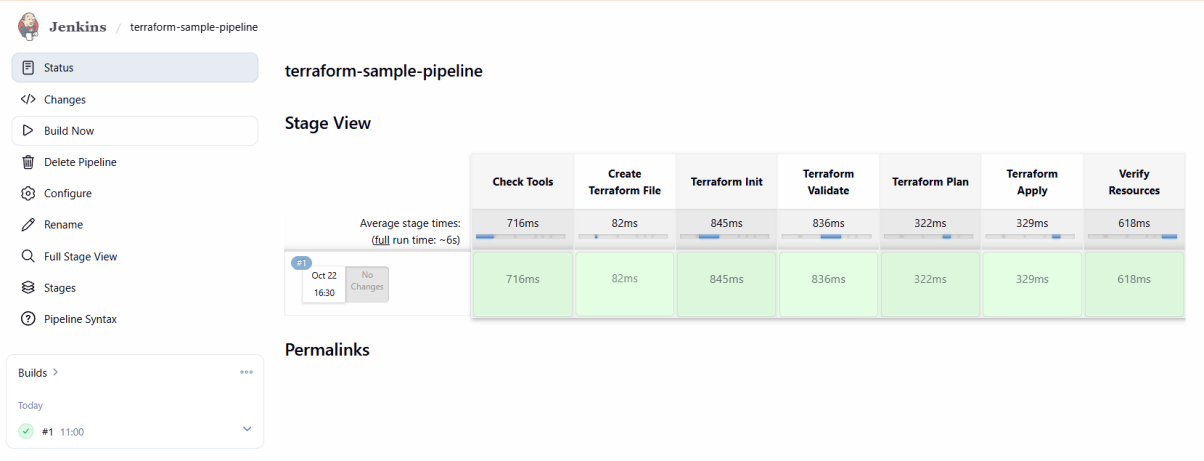
Terraform apply Apply complete! Resources: 2 added, 0 changed, 0 destroyed.

File verification i love my snupi

Terraform state
local_file.my_pet
random_pet.sunith

Final Jenkins build status Finished: SUCCESS

Implementation:



```
To perform exactly these actions, run the following command to apply:
    terraform apply "tfplan"
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Terraform Apply)
[Pipeline] sh
+ terraform apply -input=false -auto-approve tfplan
[0m[1mlocal_file.my_pet: Creating...[0m[0m
[0m[1mrandom_pet.sunith: Creating...[0m[0m
[0m[1mlocal_file.my_pet: Creation complete after 0s [id=e3796fb62ed07eee40bc9aeeaf703cbdcb730815][0m
[0m[1mrandom_pet.sunith: Creation complete after 0s [id=chintu.really.cunning.midge][0m
[0m[1m[32m
Apply complete! Resources: 2 added, 0 changed, 0 destroyed.[0m
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Verify Resources)
[Pipeline] sh
+ cat pets.txt
i love my snupi
[Pipeline] sh
+ terraform state list
local_file.my_pet
random_pet.sunith
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS
```

4. Conclusion

Terraform was successfully integrated with Jenkins on an AWS EC2 Linux instance. The Jenkins Pipeline executed the Terraform initialization, validation, planning, application, and verification stages successfully.

The Local provider created pets.txt containing the expected text, and the Random provider generated a pet-style name. Terraform state confirmed that both resources were managed successfully.

This practical demonstrated how Jenkins can automate Terraform execution through a CI/CD pipeline and established a foundation for future infrastructure automation projects.